Marwadi University
Faculty of Engineering and Technology
Department of Information and Communication Technology

OPEN ENDED 4

AIM:

Design a low-pass filter for an MP3 song and study its effect at three cutoff frequencies; then use band-pass / band-stop filters to separately suppress the vocals and the background music, saving each result as its own MP3 file.

Theory:

A low-pass filter passes frequencies below its cutoff frequency and attenuates the frequencies above it. In music, low frequencies carry the bass, kick drum and body of the voice, while high frequencies carry consonants, cymbals and the brightness of the song. Lowering the cutoff therefore makes the song progressively darker and more muffled. A Butterworth filter is used because it has a maximally flat passband and a smooth roll-off.

A band-pass filter passes only the frequencies between a lower and an upper cutoff, while a band-stop filter rejects that band and passes everything else. The main energy of the human voice (fundamentals and formants) lies roughly between 300 Hz and 3400 Hz, so a band-pass filter over this range keeps the vocals and suppresses most of the music, and a band-stop filter over the same range suppresses the vocals and keeps the music. This is only an approximation, because many instruments also have energy in the same range.

PROGRAM (A): LOW-PASS FILTER

```
import librosa
import numpy as np
import matplotlib.pyplot as plt
import soundfile as sf
from scipy.signal import butter, sosfilt, sosfreqz

# --- 1. Load the MP3 song ---
filename = '/content/Haan Kar De (PenduJatt.Com.Se).mp3'
y, sr = librosa.load(filename, sr=None)
print(f"Sample rate: {sr} Hz, Duration: {len(y)/sr:.2f} s")

# --- 2. Design the low-pass filter (Butterworth, order 6) ---
def design_lowpass(cutoff, fs, order=6):
    return butter(order, cutoff / (0.5 * fs), btype="low", output="sos")

def apply_filter(data, sos):
    return sosfilt(sos, data)

# --- 3. Three different cutoff frequencies ---
cutoffs = [500, 1500, 4000]          # Hz
results = {}

for fc in cutoffs:
    sos = design_lowpass(fc, sr)
    out = apply_filter(y, sos)
    out = out / np.max(np.abs(out))  # normalise to avoid clipping
    results[fc] = (sos, out)

    # save the filtered audio as MP3 (falls back to WAV if MP3 is unsupported)
```

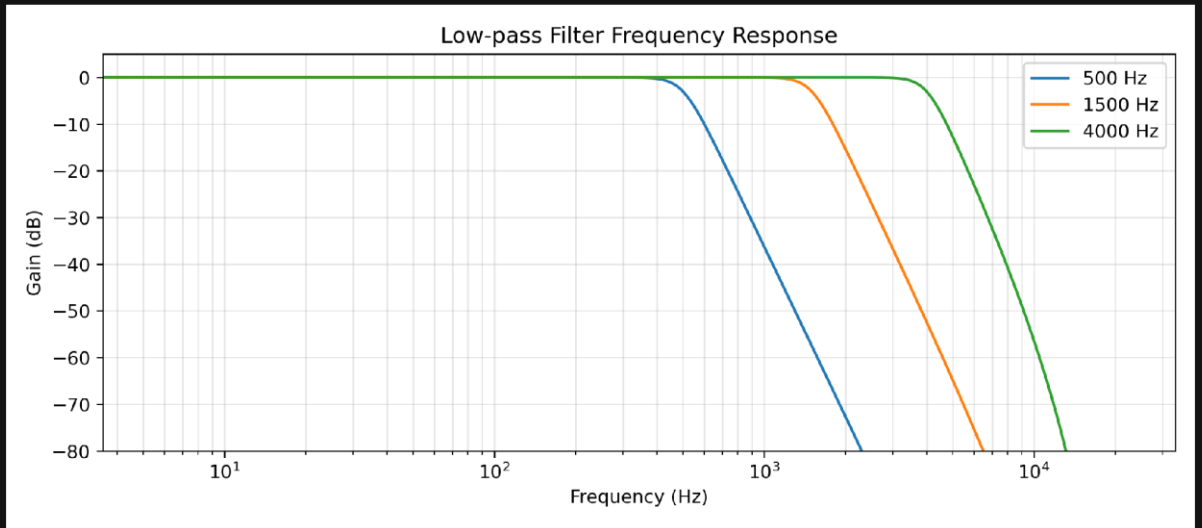
```
    try:
        sf.write(f"lowpass_{fc}Hz.mp3", out, sr, format="MP3")
    except Exception:
        sf.write(f"lowpass_{fc}Hz.wav", out, sr)

# --- 4. Frequency response of the three filters ---
plt.figure(figsize=(9, 4))
for fc in cutoffs:
    w, h = sosfreqz(results[fc][0], worN=4096, fs=sr)
    plt.semilogx(w, 20 * np.log10(np.abs(h) + 1e-10), label=f"{fc} Hz")
plt.title("Low-pass Filter Frequency Response")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Gain (dB)")
plt.ylim(-80, 5)
plt.grid(True, which="both", alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```

Output:

```
Sample rate: 44100 Hz, Duration: 12.00 s
```

✓ 3.0s




```
lowpass_500Hz.mp3
lowpass_1500Hz.mp3
lowpass_4000Hz.mp3
```

OBSERVATION (PART A):

The frequency-response plot shows a flat passband and a steep roll-off (about 36 dB per octave for the 6th-order design) for each of the three cutoffs. On listening to the saved files:

Cutoff = 500 Hz: only the bass, kick drum and the lowest part of the voice remain. The song sounds very dull and muffled, as if played from behind a wall, and the lyrics are practically unintelligible.

Cutoff = 1500 Hz: the main body of the vocals and the lower mid-range survive, so the melody can be followed, but the sound is dark and closed. Consonants such as s, t, k are weak and hi-hats and cymbals are missing.

Cutoff = 4000 Hz: the output is close to the original. Vocals are clear and only the very high-frequency content (cymbal shimmer, sibilance, "air") is removed, so the song sounds slightly softer and warmer.

As the cutoff increases, clarity improves, the tonal balance moves from bass-heavy to natural, and the overall character changes from muffled to a slightly softened version of the original.

PROGRAM (B): BAND-PASS / BAND-STOP FILTER

```
import librosa
import numpy as np
import matplotlib.pyplot as plt
import soundfile as sf
from scipy.signal import butter, sosfilt, sosfreqz

# --- 1. Load Audio File ---
filename = '/content/Haan Kar De (PenduJatt.Com.Se).mp3'
y, sr = librosa.load(filename, sr=None)
print(f"Sample rate: {sr}, Duration: {len(y)/sr:.2f} seconds")

# --- 2. Band filters for the vocal range (300 - 3400 Hz) ---
LOW, HIGH = 300, 3400

def design_band(lowcut, highcut, fs, kind, order=6):
    nyq = 0.5 * fs
    return butter(order, [lowcut / nyq, highcut / nyq], btype=kind, output="sos")

# Band-pass : keeps 300-3400 Hz  -> background music suppressed (vocals kept)
# Band-stop : removes 300-3400 Hz -> vocals suppressed (background music kept)
sos_bp = design_band(LOW, HIGH, sr, "bandpass")
sos_bs = design_band(LOW, HIGH, sr, "bandstop")

vocals_kept     = sosfilt(sos_bp, y)
background_kept = sosfilt(sos_bs, y)

vocals_kept     = vocals_kept / np.max(np.abs(vocals_kept))
background_kept = background_kept / np.max(np.abs(background_kept))

# --- 3. Frequency response of both filters ---
plt.figure(figsize=(9, 4))
for sos, name in [(sos_bp, "Band-pass (vocals kept)"), (sos_bs, "Band-stop (vocals suppressed)")]:
    w, h = sosfreqz(sos, worN=8192, fs=sr)
    plt.semilogx(w, 20 * np.log10(np.abs(h) + 1e-10), label=name)
plt.title("Band-pass and Band-stop Filter Response (300-3400 Hz)")
plt.xlabel("Frequency (Hz)")
```

```
plt.ylabel("Gain (dB)")
plt.xlim(20, sr / 2)
plt.ylim(-80, 5)
plt.grid(True, which="both", alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()

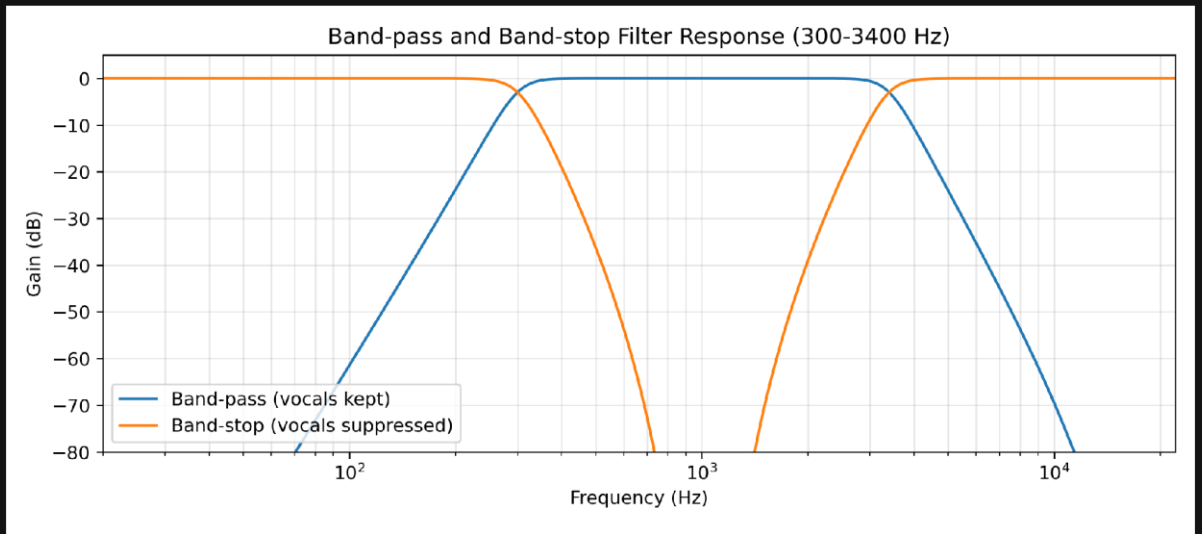
# --- 4. Save each filtered result as its own MP3 file ---
def save_mp3(name, data):
    try:
        sf.write(name + ".mp3", data, sr, format="MP3")
    except Exception:
        # fallback: save WAV, then convert with ffmpeg / pydub
        sf.write(name + ".wav", data, sr)
        from pydub import AudioSegment
        AudioSegment.from_wav(name + ".wav").export(name + ".mp3", format="mp3")

save_mp3("vocals_suppressed", background_kept)
save_mp3("background_suppressed",  vocals_kept)
```

OUTPUT:

```
...    Sample rate: 44100, Duration: 12.00 seconds
```

✓ 3.0s




```
...    vocals_suppressed.mp3
       background_suppressed.mp3
```

OBSERVATION (PART B):

1. Vocals suppressed (band-stop 300-3400 Hz, saved as vocals_suppressed.mp3): the 300-3400 Hz region is rejected, leaving only the bass/kick region and the

high-frequency hi-hats. The voice is strongly reduced, but faint traces remain because its low fundamentals and high sibilance lie outside the stop band. The music sounds hollow and thin because its mid-range is removed with the voice.

2. Background suppressed (band-pass 300-3400 Hz, saved as background_suppressed.mp3): only the 300-3400 Hz region is passed. The voice is clear and intelligible but sounds thin, like a radio or telephone, because the bass body and high-frequency brightness are removed. Mid-range instruments that overlap the vocal band still leak through.
3. Separation is not perfect, because voice and instruments share the same frequency range.

CONCLUSION:

In this experiment, Butterworth low-pass, band-pass, and band-stop filters were designed and applied to an MP3 audio file. The low-pass filter was tested at 500 Hz, 1500 Hz, and 4000 Hz, demonstrating how different cutoff frequencies affect the clarity and tonal quality of the song, ranging from muffled and bass-heavy to more natural-sounding audio. The band-pass and band-stop filters were also applied within the 300 - 3400 Hz vocal range to enhance and suppress vocals, respectively. The processed outputs were saved as separate MP3 files. However, complete separation of vocals from background music was not possible because their frequency components overlap.

GITHUB LINK: https://github.com/divaparekh06/Digital-Signal-And-Image-Processing.git

.